Cabeceras HTTP, contenido, caché y negociación — versión ampliada

La versión corta sirve para seguir las primeras observaciones de HTTPie. Esta guía amplía caché, CORS, autenticación y ejemplos de APIs; no significa que Hello Server implemente todas esas cabeceras.

Las cabeceras HTTP son metadatos que acompañan a una petición o a una respuesta. No son el contenido: explican cómo interpretar el cuerpo, cómo negociar formatos, cómo controlar caché, cómo autenticar, cómo redirigir o cómo proteger una API.

En una aplicación REST/Spring Boot no basta con mirar el JSON. Para entender una respuesta hay que leer tres piezas juntas:

1. **Estado HTTP**: qué ha pasado (`200`, `201`, `404`, `415`…).
2. **Cabeceras**: cómo interpretar o gestionar la respuesta.
3. **Cuerpo**: la representación enviada, si existe.

Lectura rápida

- **Identificar el destino:** `Host`.
- **Indicar qué formato acepta el cliente:** `Accept`.
- **Describir el formato enviado:** `Content-Type`.
- **Indicar tamaño del cuerpo:** `Content-Length`.
- **Señalar un recurso creado o una redirección:** `Location`.
- **Autenticar una petición:** `Authorization`.
- **Gestionar sesión/cookies:** `Cookie`, `Set-Cookie`.
- **Controlar caché:** `Cache-Control`, `ETag`, `Last-Modified`.
- **Hacer peticiones condicionales:** `If-None-Match`, `If-Modified-Since`, `If-Match`.
- **Informar métodos disponibles:** `Allow`.
- **Gestionar límites de peticiones:** `Retry-After`.
- **Negociar CORS:** `Origin`, `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`.

Regla práctica: **el cuerpo contiene los datos; las cabeceras dicen cómo debe tratarse ese intercambio**.

Cabeceras de petición y de respuesta

Algunas cabeceras suelen aparecer en peticiones, otras en respuestas y otras en ambas.

- `Host` — petición. Identifica el servidor lógico solicitado en HTTP/1.1.
- `Accept` — petición. Formatos de respuesta que el cliente puede procesar.
- `Content-Type` — petición y respuesta. Tipo del cuerpo enviado.
- `Content-Length` — petición y respuesta. Tamaño del cuerpo en bytes cuando se conoce.
- `Authorization` — petición. Credenciales o token enviado al servidor.
- `Cookie` — petición. Cookies que el navegador envía al servidor.
- `Set-Cookie` — respuesta. Instrucción para crear/modificar cookies en el cliente.
- `Location` — respuesta. URI relacionada: recurso creado o destino de redirección.
- `Allow` — respuesta. Métodos HTTP permitidos para un recurso.
- `Cache-Control` — petición y respuesta. Directivas de caché.
- `ETag` — respuesta. Identificador de versión de una representación.
- `If-None-Match` — petición. Condición basada en `ETag` para revalidar caché.
- `If-Match` — petición. Condición basada en `ETag` para evitar sobrescrituras.
- `Origin` — petición. Origen que realiza una petición CORS.
- `Access-Control-Allow-Origin` — respuesta. Orígenes autorizados en CORS.

No todas aparecen siempre. Una API sencilla puede no usar caché, cookies ni CORS explícito al principio, pero hay que reconocerlas cuando aparezcan en el navegador, HTTPie, Postman o los tests.

`Accept` frente a `Content-Type`

Esta distinción es FUNDAMENTAL. Mucha gente la mezcla porque ambas mencionan formatos.

`Accept`: qué quiero recibir

`Accept` lo envía el cliente para expresar qué formatos sabe procesar o prefiere.

```
GET /api/tasks/7 HTTP/1.1
Host: localhost:8080
Accept: application/json
```

El cliente está diciendo: «si puedes, respóndeme con JSON».

`Content-Type`: qué estoy enviando

`Content-Type` describe el cuerpo que viaja en esa petición o respuesta.

```
POST /api/tasks HTTP/1.1
Host: localhost:8080
Content-Type: application/json

{"title":"Repasar cabeceras"}
```

El cliente está diciendo: «el cuerpo que te envío es JSON».

La respuesta también puede tener `Content-Type`:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/tasks/7

{"id":7,"title":"Repasar cabeceras"}
```

Ahí el servidor está diciendo: «el cuerpo que te devuelvo es JSON».

Error típico

No tiene sentido enviar un cuerpo JSON con `Content-Type: text/plain` y esperar que Spring Boot lo trate como JSON.

```
POST /api/tasks HTTP/1.1
Content-Type: text/plain

{"title":"Esto parece JSON"}
```

El texto del cuerpo parece JSON, pero la cabecera dice `text/plain`. El servidor puede responder `415 Unsupported Media Type` si el endpoint espera `application/json`.

Tipos de contenido habituales

- `text/html`: página HTML para navegador.
- `text/plain`: texto plano.
- `application/json`: JSON en APIs REST.
- `application/xml`: XML.
- `application/x-www-form-urlencoded`: formularios HTML tradicionales.
- `multipart/form-data`: formularios con archivos.
- `application/problem+json`: errores estructurados según Problem Details.

Que una ruta contenga `/api` no convierte automáticamente la respuesta en JSON. Lo confirma `Content-Type`.

Cabeceras importantes en APIs REST

`Location`

`Location` señala una URI relacionada. En REST es especialmente importante tras crear un recurso.

```
HTTP/1.1 201 Created
Location: /api/tasks/7
Content-Type: application/json

{"id":7,"title":"Nueva tarea"}
```

En Spring Boot:

```
URI location = URI.create("/api/tasks/" + saved.id());
return ResponseEntity.created(location).body(saved);
```

Si devuelves `201 Created` pero no das forma de localizar el recurso creado, la respuesta queda coja.

`Allow`

`Allow` puede aparecer en respuestas `405 Method Not Allowed` para indicar qué métodos sí están permitidos.

```
HTTP/1.1 405 Method Not Allowed
Allow: GET, POST
```

No significa que toda la aplicación acepte esos métodos: se refiere al recurso concreto.

`Authorization`

`Authorization` envía credenciales. En APIs modernas suele llevar un token:

```
Authorization: Bearer eyJhbGciOi...
```

No se debe copiar en capturas, foros, entregas ni issues. Si hace falta mostrar la cabecera, se redacta así:

```
Authorization: Bearer <REDACTED>
```

`Cookie` y `Set-Cookie`

`Set-Cookie` lo envía el servidor para que el cliente guarde una cookie. `Cookie` lo envía después el cliente en peticiones posteriores.

```
HTTP/1.1 200 OK
Set-Cookie: JSESSIONID=abc123; HttpOnly; Secure; SameSite=Lax
```

Flags importantes:

- **`HttpOnly`:** evita acceso desde JavaScript del navegador.
- **`Secure`:** solo se envía por HTTPS.
- **`SameSite`:** reduce ciertos ataques CSRF.
- **`Max-Age` / `Expires`:** controla duración de la cookie.

Una cookie de sesión es sensible. Hay que tratarla como una contraseña temporal.

Caché HTTP

La caché evita transferir de nuevo representaciones que el cliente ya tiene. Es muy útil en CSS, JavaScript, imágenes y recursos que cambian poco. En APIs REST también puede ayudar, pero hay que diseñarla con cuidado.

`Cache-Control`

`Cache-Control` indica cómo se puede almacenar o reutilizar una respuesta.

- **`no-store`:** no guardar la respuesta en caché.
- **`no-cache`:** se puede guardar, pero hay que revalidar antes de reutilizar.
- **`max-age=60`:** se puede reutilizar durante 60 segundos.
- **`public`:** puede almacenarse en cachés compartidas.
- **`private`:** solo caché privada del usuario.
- **`must-revalidate`:** si caduca, debe revalidarse antes de usarse.

Ejemplos:

```
Cache-Control: no-store
```

Adecuado para datos sensibles.

```
Cache-Control: public, max-age=3600
```

Adecuado para recursos públicos que cambian poco.

`ETag` e `If-None-Match`

`ETag` identifica una versión concreta de una representación.

Primera respuesta:

```
HTTP/1.1 200 OK
Content-Type: application/json
ETag: "task-7-v3"

{"id":7,"title":"Repasar cabeceras","version":3}
```

Petición posterior:

```
GET /api/tasks/7 HTTP/1.1
If-None-Match: "task-7-v3"
```

Si no ha cambiado:

```
HTTP/1.1 304 Not Modified
ETag: "task-7-v3"
```

`304` no lleva el cuerpo completo de nuevo. El cliente reutiliza lo que ya tenía.

`Last-Modified` e `If-Modified-Since`

Funcionan con fechas en lugar de identificadores de versión.

```
Last-Modified: Tue, 06 Oct 2026 08:30:00 GMT
```

```
If-Modified-Since: Tue, 06 Oct 2026 08:30:00 GMT
```

`ETag` suele ser más preciso porque una fecha puede no capturar todos los cambios o tener problemas de resolución temporal.

Condiciones para evitar sobrescrituras

Las cabeceras condicionales no sirven solo para caché. También pueden evitar que dos clientes sobrescriban cambios sin darse cuenta.

Flujo típico:

1. El cliente obtiene un recurso con `ETag: "v3"`.
2. El usuario edita datos.
3. El cliente envía el cambio con `If-Match: "v3"`.
4. El servidor solo actualiza si el recurso sigue en esa versión.

```
PUT /api/tasks/7 HTTP/1.1
Content-Type: application/json
If-Match: "v3"

{"title":"Nuevo título"}
```

Si otra persona ya lo cambió y ahora la versión es `"v4"`, el servidor puede responder:

```
HTTP/1.1 412 Precondition Failed
```

Esto conecta con control de concurrencia optimista, que se verá con más detalle en aplicaciones reales.

CORS: cabeceras que el navegador exige

CORS aparece cuando una página cargada desde un origen intenta llamar a una API de otro origen.

Origen significa combinación de:

- esquema: `http` o `https`;
- host: `localhost`, `api.example.com`;
- puerto: `3000`, `8080`.

Ejemplo: una app frontend en `http://localhost:5173` llama a una API en `http://localhost:8080`. Para el navegador son orígenes distintos.

Petición:

```
GET /api/tasks HTTP/1.1
Origin: http://localhost:5173
```

Respuesta permitiendo ese origen:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5173
Content-Type: application/json
```

En peticiones no simples, el navegador puede enviar antes una petición `OPTIONS` de preflight:

```
OPTIONS /api/tasks HTTP/1.1
Origin: http://localhost:5173
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type, authorization
```

Respuesta posible:

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: content-type, authorization
```

CORS no es autenticación. Solo es una política del navegador para decidir si una página puede leer la respuesta de otro origen.

Cabeceras de seguridad que conviene reconocer

No hace falta dominarlas en UD1, pero sí reconocerlas cuando aparezcan.

- `Strict-Transport-Security`: obliga a usar HTTPS durante un tiempo.
- `Content-Security-Policy`: limita desde dónde se pueden cargar scripts, estilos, imágenes y otros recursos.
- `X-Content-Type-Options: nosniff`: evita que el navegador adivine tipos de contenido.
- `X-Frame-Options`: controla si la página puede cargarse en un `iframe`.
- `Referrer-Policy`: controla cuánta información de origen se envía al navegar.

Estas cabeceras no sustituyen a programar bien, pero reducen superficie de ataque en aplicaciones web.

Cómo leer una traza HTTP

Ejemplo de petición:

```
POST /api/tasks HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <REDACTED>

{"title":"Repasar HTTP"}
```

Lectura:

- `POST /api/tasks`: se pide procesar/crear algo en esa colección.
- `Accept: application/json`: el cliente quiere recibir JSON.
- `Content-Type: application/json`: el cuerpo enviado es JSON.
- `Authorization`: hay credenciales; no deben compartirse.
- La línea en blanco separa cabeceras y cuerpo.

Ejemplo de respuesta:

```
HTTP/1.1 201 Created
Location: /api/tasks/7
Content-Type: application/json
Cache-Control: no-store

{"id":7,"title":"Repasar HTTP"}
```

Lectura:

- `201 Created`: se creó un recurso.
- `Location`: URI del recurso nuevo.
- `Content-Type`: el cuerpo devuelto es JSON.
- `Cache-Control: no-store`: no se debe guardar en caché.

Errores frecuentes

- **Confundir `Accept` con `Content-Type`:** uno expresa lo que quiero recibir; el otro describe lo que envío.
- **Pensar que `/api` siempre devuelve JSON:** lo confirma `Content-Type`, no el nombre de la ruta.
- **Compartir `Authorization` o cookies reales:** equivale a filtrar credenciales.
- **Usar caché sin pensar en datos sensibles:** puede exponer información privada.
- **Creer que CORS protege la API:** CORS limita al navegador; la API sigue necesitando autenticación y autorización.
- **Ignorar `Location` en un `201 Created`:** el cliente pierde una forma clara de localizar el recurso creado.

Exploración con HTTPie y navegador

Con Hello Server arrancado:

```
http --headers GET http://localhost:8080/
http --body GET http://localhost:8080/
http --headers GET http://localhost:8080/api/hello
http --body GET http://localhost:8080/api/hello
```

Compara:

1. `Content-Type` de `/` y `/api/hello`.
2. Qué parte es cabecera y qué parte es cuerpo.
3. Si aparece alguna cabecera relacionada con caché.
4. Qué muestra el navegador en **F12 → Red → Encabezados / Respuesta**.

No compartas cookies, tokens ni `Authorization` reales. Si necesitas enseñar una traza, reemplaza los valores sensibles por `<REDACTED>`.

Comprobación

Para cada caso, indica qué cabecera mirarías o enviarías:

1. Quieres confirmar si una respuesta de `/api/hello` es JSON.
2. Envías un `POST` con JSON a una API REST.
3. Un `POST` crea `/api/tasks/7` y quieres comunicar la URI nueva.
4. Quieres evitar descargar de nuevo una representación que no ha cambiado.
5. Quieres impedir que dos clientes sobrescriban el mismo recurso sin detectar el conflicto.
6. Vas a compartir una captura de una petición autenticada.
7. Un frontend en `localhost:5173` llama a una API en `localhost:8080`.

La respuesta correcta no es memorizar nombres: es explicar qué problema resuelve cada cabecera y qué riesgo evita.